DIFFUSIONART: EFFICIENT, STABLE AND SCALABLE REINFORCEMENT OF DIFFUSION MODELS

Zhenglin Wan[1,2] **Jiayu Chen**[2,†] **Zhengfeiyang Zhang**[1] **Pengfei Zhou**[1] **Wangbo Zhao**[3]
Fangyuan Kong[2] **Pengfei Wan**[2] **Kun Gai**[2] **Yang You**[1]
[1]HPC-AI Lab, National University of Singapore [2]Kling AI [3]HKUST
Email: vanzl@u.nus.edu, jia768167535@gmail.com
GitHub: https://github.com/vanzll/DiffusionART
Project Page: https://vanzll.github.io/diffusionart-project/

ABSTRACT

Online reinforcement learning (RL) fine-tunes diffusion models with black-box rewards, but reverse-process methods such as Flow-GRPO are limited in efficiency by poor credit assignment, while forward-process methods such as DiffusionNFT are efficient yet unstable without carefully tuned exponential-moving-average (EMA) off-policy training. We show that NFT-style objectives contain **a hidden forward–reverse consistency force** that joint positive–negative training cancels only partially, and that EMA stabilizes training through **restoration** rather than off-policy rollouts. We introduce **DiffusionART** (Diffusion Reinforcement with Aggregation-First Reward Tilting), which aggregates each prompt's rewards into a shared reward-improving direction for reverse-process states, removing the consistency force by construction, and adds a trust-region-masked second update for drift-adaptive restoration without EMA tuning. It requires no differentiable reward model, remains effective with fewer samples per prompt, and supports pairwise reward models (DiffusionART-Pairwise). Across image and video generation, DiffusionART trains efficiently and stably without per-task tuning. In particular, on HunyuanVideo-13B with equally weighted visual and motion quality rewards, DiffusionART reaches the late-stage mean reward levels of DiffusionNFT and Flow-GRPO-Fast with 12.02× and 11.72× fewer GPU hours, respectively, while achieving 28.8% and 53.0% higher mean training rewards.

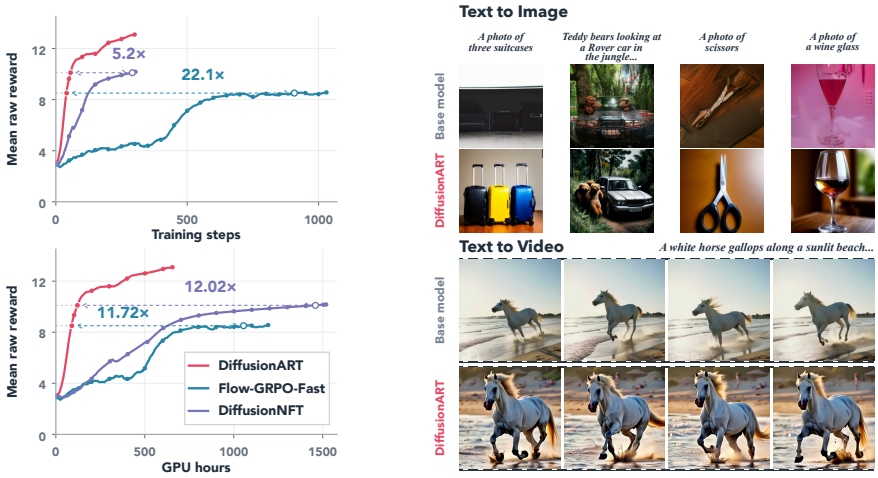

Figure 1: **Efficient learning and improved generations. Left:** Mean of raw VQ and MQ (equal weights). **Right:** Selected same-prompt images and video frames before and after DiffusionART.

[†]Corresponding author.

1 Introduction

Visual diffusion and flow-matching models[1] are typically trained to fit the data distribution by regressing noise or velocity targets rather than to optimize downstream criteria such as visual quality, motion quality, or human preference, making reward-guided fine-tuning central to recent image and video generation tasks [2,3]. We focus on online diffusion Reinforcement Learning (RL), which, unlike differentiable reward-model optimization such as ReFL [4] or offline preference learning such as Diffusion-DPO [5], evaluates clean samples from the evolving generator and can use black-box rewards. Because these rewards arrive only after a complete output is generated, online diffusion RL faces a credit-assignment problem: costly output-level feedback must be translated into effective updates across many denoising states [6,7].

Within this paradigm, *reverse-process RL* [8], such as Flow-GRPO and its variants [6,9,10,11], optimizes sampled denoising transitions via group-based endpoint reward feedback. It trains on states actually visited by the generator but requires ODE-to-SDE conversion for tractable likelihoods, and its policy gradient updates each selected transition along its own transition-level sampling noise, weighted by a trajectory-level advantage that this transition only partly determines, resulting in poor credit assignment and limiting training efficiency. *Forward-process RL*, such as DiffusionNFT [7], instead applies forward noising to the generated clean samples and regresses velocities with reward-weighted positive and negative objectives. It needs no transition likelihoods and is highly efficient, but it is unstable without two stabilizers: joint positive–negative training and an exponential moving average (EMA) rollout policy whose lag must be tuned for each task [7]. Neither family is thus both efficient and stable without per-task tuning. In this paper, we focus on studying the instability of forward-process RL and developing a scalable algorithm which is both stable and efficient.

We first examine NFT-style training, starting from a puzzle: its positive and negative branches share the same ideal reward-improving direction, so either alone should suffice, yet each alone collapses (Figure 2(a)). We find that NFT's targets, built from forward-noised endpoints, implicitly assume that the model's sampling velocity equals the ideal forward velocity induced by its own endpoints. The two differ in practice, and the gap introduces a hidden non-reward-improving *forward–reverse consistency force*: the positive branch pulls each state's velocity toward the ideal forward velocity and the negative branch pushes it away. Joint training can cancel this force but only partially, since reward centering balances rewards across samples rather than at each state. The residual can destabilize training and slow the reward optimization. We further show that EMA stabilizes NFT through the *restoration force* of its lagged reference, not through off-policy rollouts, but the EMA trick must be tuned per task.

Driven by these insights, we introduce **DiffusionART** (Diffusion Reinforcement with Aggregation-First Reward Tilting). It borrows supervised velocity regression from forward-process training but applies updates to states and actions during reverse-process rollouts. It first aggregates each prompt's samples and rewards into a shared reward-improving direction $G(c)$, then regresses each visited state's action toward its guided target (Section 4.1). As this target never involves the forward velocity, the consistency force is removed by construction rather than by cross-sample cancellation, and each state receives an explicit reward-improving direction instead of a scalar advantage passed through likelihood ratios. A second update with a trust-region mask mechanism on each rollout adds a restoration force proportional to each state's drift, replacing EMA with no schedule to tune (Section 4.2). We summarize our main contributions as follows:

Main Contributions

1. Diagnosing the instability of NFT-style training. The NFT objective implicitly contains a harmful forward–reverse consistency force that joint training cancels only partially (Section 3.1).

2. Identifying restoration as the key stabilizer. EMA stabilizes NFT through restoration rather than off-policy data, but requires per-task tuning (Section 3.2).

3. Aggregation-first reward tilting with adaptive restoration. DiffusionART removes the consistency force by construction, eliminating the need for EMA with drift-adaptive restoration (Section 4).

4. Efficient and stable training. DiffusionART matches the strongest baseline's efficiency on SD3.5-M without EMA or per-task tuning. Under VQ:MQ $=$ 1:1 weights on HunyuanVideo-13B, it reaches the late-stage mean VQ and MQ rewards of DiffusionNFT and Flow-GRPO-Fast with $12.02\times$ and $11.72\times$ fewer GPU hours, while achieving 28.8% and 53.0% higher mean training rewards, respectively. For image-to-video generation, it reaches MQ 6 with $4.8\times$ fewer GPU hours than Flow-GRPO-Fast (Section 5).

[1]We use "diffusion" as a general term. Our paper targets rectified-flow with linear data–noise interpolation [1], which shares the core principles of diffusion.

2 Preliminaries

Diffusion RL [12] includes reverse-process policy optimization, such as Flow-GRPO [6], and forward-process optimization, such as DiffusionNFT [7,13,14]. We focus on DiffusionNFT and defer Flow-GRPO to Appendix C.1. At each iteration, for each prompt c, DiffusionNFT samples K clean images or videos x_0 from a frozen rollout policy $p_{\text{old}}(x_0 \mid c)$; we call these generated samples *endpoints*. DiffusionNFT maps each raw reward $R_i := R(x_0^{(i)}, c)$ to a *goodness* weight $r_i := r(x_0^{(i)}, c) = \frac{1}{2}[1 + \text{clip}((R_i - \bar{R}_c)/Z_c, -1, 1)] \in [0, 1]$, where $\bar{R}_c = K^{-1}\sum_i R_i$ is the prompt's mean reward and $Z_c > 0$ controls the scale; without clipping, $\sum_i (2r_i - 1) = 0$. Weighting p_{old} by r and $1 - r$ induces the positive and negative endpoint distributions $p_+(x_0 \mid c) \propto r(x_0, c)p_{\text{old}}(x_0 \mid c)$ and $p_-(x_0 \mid c) \propto [1 - r(x_0, c)]p_{\text{old}}(x_0 \mid c)$. DiffusionNFT is motivated by the result that the ideal velocities induced by forward noising the positive, original, and negative endpoint distributions are collinear. Moving toward the positive field and away from the negative field therefore follows the same ideal improvement direction.

We parameterize generation from $t = 1$ (noise) to $t = 0$ (clean samples), so velocity points along the denoising direction. For a forward state $x_t = \alpha_t x_0 + \sigma_t \epsilon$, let $u_t = -\dot{\alpha}_t x_0 - \dot{\sigma}_t \epsilon$ denote its sample velocity target, where dots denote derivatives with respect to t; standard rectified-flow interpolation [1,15] gives $u_t = x_0 - \epsilon$. Let $v_{\text{old}} := v_{\theta_{\text{old}}}(x_t, t, c)$ and $v_\theta(x_t, t, c)$ be the velocities predicted by the rollout policy p_{old} (frozen within an iteration) and the trainable policy, respectively, and let $\mathcal{D}_{\text{old}}$ be the training distribution of (c, x_0, r, t, ϵ) with $x_0 \sim p_{\text{old}}(\cdot \mid c)$ and $\epsilon \sim \mathcal{N}(0, I)$. For $\beta > 0$, DiffusionNFT constructs $v_{\theta,\pm} = v_{\text{old}} \pm \beta(v_\theta - v_{\text{old}})$ and minimizes $\mathcal{L}_{\text{NFT}} = \mathbb{E}_{\mathcal{D}_{\text{old}}}[r\|v_{\theta,+} - u_t\|_2^2 + (1 - r)\|v_{\theta,-} - u_t\|_2^2]$. Equivalently, let $a = (2r - 1)/\beta$ and define the frozen target $v_{\text{tgt}} = au_t + (1 - a)v_{\text{old}}$. Then

$$\mathcal{L}_{\text{NFT}} = \beta^2 \mathbb{E}_{\mathcal{D}_{\text{old}}}\left[\|v_\theta - v_{\text{tgt}}\|_2^2\right] + \text{const}, \tag{1}$$

where const is independent of θ. Notably, NFT retains a v_{old} anchor in v_{tgt}. At the on-policy initialization $v_\theta = v_{\text{old}}$, each sample's output-space descent direction is proportional to $(2r - 1)(u_t - v_{\text{old}})$, so high- and low-reward samples drive the same regression residual in opposite directions (see Appendix B.2.1 for both derivations). After optimizing the rollout batch, DiffusionNFT updates the rollout policy by exponential moving average (EMA): $\theta_{\text{old}} \leftarrow \eta\theta_{\text{old}} + (1 - \eta)\theta$.

3 Understanding NFT-style Diffusion Reinforcement Learning

DiffusionNFT is highly efficient, but its stability relies on joint positive–negative training and an off-policy EMA rollout policy [7]. We examine why both are needed and where they fall short: joint training only partially cancels a hidden forward–reverse consistency force, while EMA provides restoration whose strength requires task-specific tuning. These limitations motivate the two components of DiffusionART in Section 4.

3.1 Joint Positive–Negative Training: Partial Cancellation of a Hidden Consistency Force

We first remove EMA to isolate joint training, so $v_\theta = v_{\text{old}}$ at every on-policy update. Consistent with Zheng et al. [7], either branch alone collapses, whereas joint training learns effectively (Figure 2(a)). This is puzzling: the ideal positive and negative guidance directions are collinear, suggesting that either branch alone should suffice.

To make this precise, for $b \in \{+, \text{old}, -\}$, let P_b denote the endpoint distribution $p_b(x_0 \mid c)$, and let Q_b denote the joint **forward** distribution of $(x_0, \epsilon, x_t, u_t)$ obtained by drawing $x_0 \sim P_b$ and $\epsilon \sim \mathcal{N}(0, I)$ independently and forming x_t and $u_t = x_0 - \epsilon$ by forward noising. With $s_t = (x_t, t, c)$, the *ideal forward velocity* induced by P_b is $v_{\text{fwd}}^b(s_t) := \mathbb{E}_{Q_b}[u_t \mid s_t]$. We write $v_{\text{fwd}}^{\text{old}}$ for $b = \text{old}$, and $v_{\text{rvs}}^{\text{old}} := v_{\text{old}}$ for the field that the reverse sampler actually uses to generate the endpoint distribution.

Lemma 1 (Ideal NFT guidance is conditional reward covariance)**.** *At a fixed state s_t, let Q_{old} denote Q_b with $b = \text{old}$, and let $q := \mathbb{E}_{Q_{\text{old}}}[r(x_0, c) \mid s_t]$ and $g := \text{Cov}_{Q_{\text{old}}}[r(x_0, c), u_t \mid s_t]$. If the tilted distributions and moments exist and $0 < q < 1$, then*

$$q\big(v_{\text{fwd}}^+ - v_{\text{fwd}}^{\text{old}}\big) = (1 - q)\big(v_{\text{fwd}}^{\text{old}} - v_{\text{fwd}}^-\big) = g, \tag{2}$$

where all velocities are evaluated at s_t. Hence the positive and negative branches point along the same reward-tilt direction g.

Appendix B.1 provides the proof. The ideal guidance of the two branches thus differs only in step size: under *forward–reverse compatibility*, i.e., if the reverse sampler satisfied $v_{\text{rvs}}^{\text{old}} = v_{\text{fwd}}^{\text{old}}$, either branch alone would

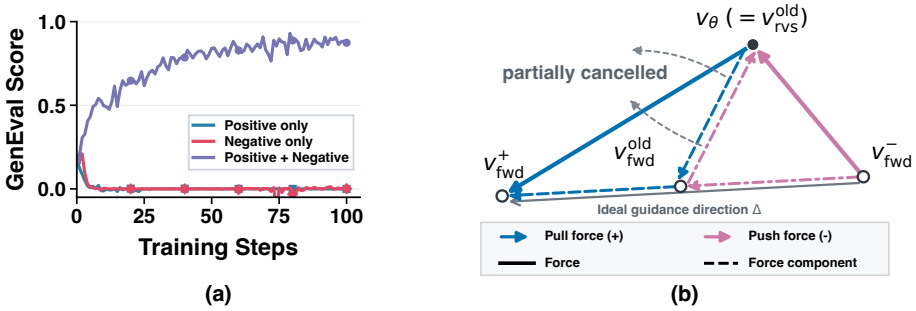

Figure 2: (a) Ablation results for positive and negative objectives. (b) The positive and negative objectives partially cancel the consistency force.

suffice. In practice, however, the reverse sampling field generally differs from the ideal forward velocity induced by its own endpoints. We denote this forward–reverse gap by $e := v^{\text{old}}_{\text{fwd}}(s_t) - v^{\text{old}}_{\text{rvs}}(s_t)$. Proposition 1 shows how it enters the NFT update.

Proposition 1 (Practical forward-process RL contains a forward–reverse consistency force)**.** *Let $\mathcal{L}_{\text{NFT}}(s_t)$ denote Equation 1 conditioned on s_t, q and g follow the definitions of Lemma 1. At the pure on-policy update point $v_\theta = v^{\text{old}}_{\text{rvs}}$ and for $\beta > 0$, its normalized output-space force is*

$$\mathcal{F}_{\text{NFT}}(s_t) := -\frac{1}{2\beta}\,\nabla_{v_\theta(s_t)}\mathcal{L}_{\text{NFT}}(s_t)\big|_{v_\theta = v^{\text{old}}_{\text{rvs}}} = 2g + (2q-1)e. \tag{3}$$

Why both branches are needed. Proposition 1 decomposes the NFT force into reward-improving guidance and a self-consistency force weighted by the statewise reward imbalance $2q - 1$. The individual branch forces are $\mathcal{F}_+ = g + qe$ and $\mathcal{F}_- = g - (1-q)e$, which sum to $\mathcal{F}_{\text{NFT}}$ (proof in Appendix B.2). Each branch alone thus carries an uncancelled consistency force: the positive branch pulls the velocity toward $v^{\text{old}}_{\text{fwd}}$ with weight q, whereas the negative branch pushes it away with weight $1 - q$. Joint training therefore does more than double the guidance: the branches reinforce each other along g but oppose each other along e, leaving $(2q-1)e$. Without clipping, prompt-level reward centering makes q average to $1/2$ across states, allowing partial cancellation after gradient aggregation (Figure 2(b)).

Why joint training is not enough. For nonzero consistency gaps, cancellation is exact only if $q = 1/2$ at every state, which prompt-level normalization does not guarantee: $\sum_i (2r_i - 1) \approx 0$ neither balances each state nor implies $\sum_i (2r_i - 1)e_i \approx 0$, where e_i denotes the one-sample Monte Carlo estimate of e. States with $q > 1/2$ are systematically pulled toward $v^{\text{old}}_{\text{fwd}}$, while those with $q < 1/2$ are pushed away. At $t = 1$, the state contains no endpoint information, so without clipping $q = 1/2$ and the residual vanishes. As noise decreases, states reveal more endpoint quality, making q more extreme and increasing the weight $|2q - 1|$ on the gap. The residual dilutes reward-improving guidance in the aggregate gradient and requires an additional stabilizer (Section 3.2).

Insight 1: Joint training cancels the consistency force only partially

In NFT-style forward-process training, either branch alone carries an uncancelled forward–reverse consistency force that can destabilize training; joint training only partially cancels it across samples, not at each state. A better design should remove this force statewise rather than rely on cross-sample cancellation (Section 4.1).

3.2 Off-Policy EMA: Stability through Restoration, at the Cost of Task-Specific Tuning

Even with joint training, pure on-policy NFT collapses after a sharp rise in its pre-clipping parameter-gradient norm (Figure 3(a)). EMA may stabilize training through (1) a slower-moving rollout distribution or (2) the lagged reference v_{old} retained in the loss (in Eq. 1, the v_{tgt} contains v_{old}, rather than v_θ). We show that the latter, namely restoration toward this reference, is the key stabilizing mechanism. Let $d = v_\theta - v_{\text{old}}$ denote the current policy's drift from the rollout old policy and $h = u_t - v_{\text{old}}$ the sample training residual.

Proposition 2 (EMA brings an explicit restoring term to DiffusionNFT)**.**

$$\mathcal{L}_{\text{NFT}} = \mathbb{E}_{\mathcal{D}_{\text{old}}}\left[-2\beta(2r-1)\langle d, h\rangle + \beta^2\|d\|_2^2\right] + \text{const}, \tag{4}$$

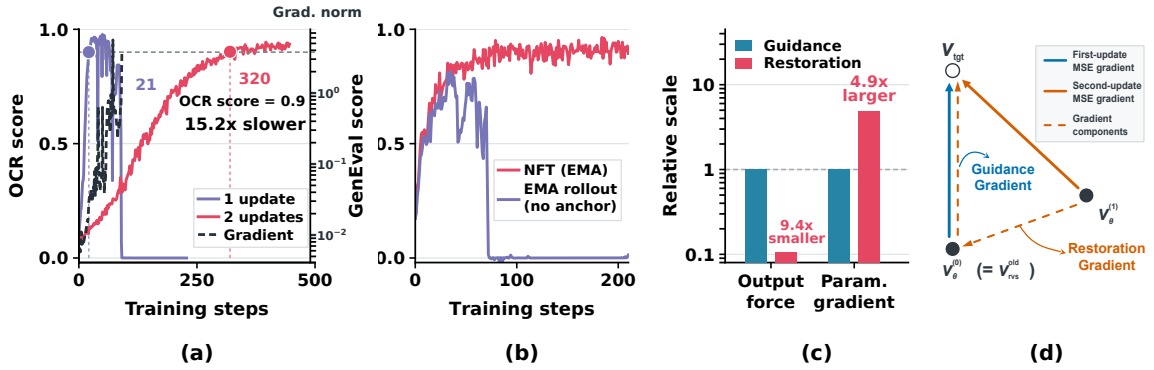

Figure 3: (a) On-policy DiffusionNFT without EMA ($\beta = 1$): increasing number of micro-batches per rollout from 1 to 2 results in $15.2\times$ slower training. The dashed curve is the one-update run's gradient norm, spiking as the reward collapses. (b) EMA-policy rollouts without the old-policy anchor still collapse. (c) At the second update, averaged over rollouts 0–17, restoration has a $9.4\times$ smaller output-force RMS but a $4.9\times$ larger parameter-gradient norm than guidance. (d) The second MSE update decomposes into guidance and restoration.

where const is independent of the trainable parameters.

Why EMA stabilizes. Proposition 2 separates the NFT loss into a linear term carrying both reward guidance and the consistency force, and a quadratic term restoring v_θ toward v_{old} (proof in Appendix B.2). At $d = 0$, the restoration gradient vanishes. Pure on-policy NFT therefore receives no restoration, leaving the residual consistency force unopposed. EMA maintains a nonzero drift from the lagged reference, allowing restoration to counteract this residual.

We test these explanations with two ablations. First, retaining EMA-policy rollouts but removing the old-policy anchor from the gradient (i.e. replacing v_{old} by v_θ in v_{tgt} of Eq 1) still causes collapse (Figure 3(b)): off-policy data alone does not stabilize NFT. Second, without EMA, we split each rollout batch into two disjoint minibatches and update once on each, using the original objective with $\beta = 1$. The first update starts at $v_\theta^{(0)} = v_{\text{old}}$, where restoration is zero. After the model moves to $v_\theta^{(1)}$, the second update combines guidance with restoration toward $v_\theta^{(0)}$ (Figure 3(d)). We surprisingly found that this two-update run avoids collapse over the observed training window without EMA, but significantly slows training by 15.2x (Figure 3(a)), supporting restoration as the key stabilizing mechanism at the cost of efficiency in NFT-style forward-process training.

Why EMA is not enough. EMA controls restoration indirectly through the reference policy's lag: too little restoration risks collapse, while too much slows reward improvement. Its schedule therefore requires task-specific tuning. DiffusionNFT's results illustrate this sensitivity [7]: fully on-policy updates collapse, nearly offline updates converge impractically slowly, and the default schedule $\eta = \min(0.001i, 0.5)$, where i is the iteration, is unstable on OCR, which requires a maximum retention of 0.999. In our FLUX experiments, transferring NFT's GenEval-tuned EMA and β settings also leads to unstable training and can cause collapse (Figure 6(a)). Such tuning requirement limits scalability.

Two NFT updates per rollout provide restoration without EMA (Figure 3(d)), but significantly slow training (Figure 3(a)). Counterintuitively, averaged over rollouts 0–17, restoration has a $9.4\times$ smaller output-force RMS than guidance, yet its aggregated parameter-gradient norm is $4.9\times$ larger (Figure 3(c)). This suggests that heterogeneous per-sample guidance largely cancels across states, while restoration survives aggregation and dominates the update, slowing reward optimization. We leave a detailed analysis to our future work.

Insight 2: Restoration is crucial for stabilizing NFT-style forward-process training

Restoration stabilizes NFT-style training, but EMA requires task-specific tuning, while two updates per rollout avoid EMA tuning at the cost of slower reward improvement. A better design should adapt restoration to each state's drift without task-specific tuning or overwhelming reward guidance (Section 4.2).

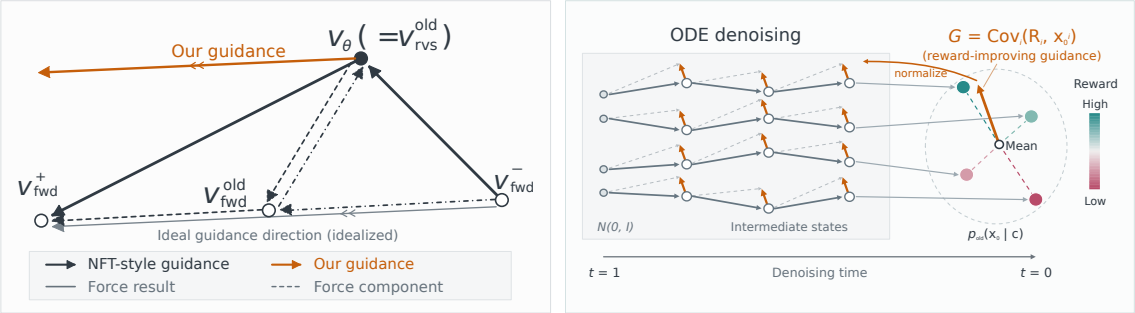

Figure 4: **Aggregation-first guidance. Left:** NFT's positive and negative branches introduce opposing consistency forces that may not fully cancel (Section 3.1). DiffusionART avoids these forces by applying $G(c)$ directly at the reverse velocity. **Right:** We aggregate each prompt's endpoints and rewards into normalized guidance $G(c)$ (Equation 5) and apply it to states visited along this rollout.

4 DiffusionART: Aggregation-first Reward Tilting with Adaptive Restoration

DiffusionART (Algorithm 1) combines the advantages of forward- and reverse-process RL, while avoiding their limitations. It applies aggregation-first reward guidance to reverse-rollout states (section 4.1) and uses a second MSE update for drift-adaptive restoration (section 4.2), addressing the two failure modes identified in Section 3. A trust-region mechanism for an ODE solver further avoids excessive policy updates (section 4.3).

4.1 Aggregation-First Guidance

The consistency force enters NFT because its regression target is built from the forward velocity $v_{\text{fwd}}^{\text{old}}$, which differs from the reverse field by e (Proposition 1). We instead aggregate each prompt's endpoints and rewards alone into guidance for reverse-rollout states. At each iteration, we exponentially tilt the current rollout distribution, rather than a fixed pretrained distribution [13,16]: $p_\tau(x_0 \mid c) \propto p_{\text{old}}(x_0 \mid c) \exp\big(\tau R(x_0, c)\big)$. Here $\tau \geq 0$ controls the preference for higher-reward endpoints, and $\tau = 0$ recovers p_{old}. The first-order change in the endpoint mean is a covariance:

Lemma 2 (Reward tilting moves the endpoint mean along the reward–endpoint covariance)**.** *Under standard integrability conditions, the derivative of the tilted endpoint mean at $\tau = 0$ equals the reward–endpoint covariance:* $\frac{\partial}{\partial \tau} \mathbb{E}_{p_\tau}[x_0 \mid c]\big|_{\tau=0} = \text{Cov}_{p_{\text{old}}}(R, x_0 \mid c)$.

Lemma 2, proved in Appendix B.3, shows that up-weighting higher-reward endpoints shifts their mean along $\text{Cov}_{p_{\text{old}}}(R, x_0 \mid c)$. Under rectified flow, $u_t = x_0 - \epsilon$ with ϵ independent of x_0, so $\text{Cov}_{Q_{\text{old}}}(R, u_t \mid c) = \text{Cov}_{p_{\text{old}}}(R, x_0 \mid c)$ at every noise level. This direction is the prompt-level counterpart of ideal NFT guidance g (Lemma 1): without clipping, r is affine in R, and at $t = 1$, when the forward state reveals no endpoint information, g equals this covariance up to a positive factor. We estimate the covariance from each prompt's K rollout endpoints and rewards, then normalize it to obtain $G(c)$:

$$\boxed{\overline{\text{Cov}}(R, x_0 \mid c) = \frac{1}{K} \sum_{i=1}^{K} (R_i - \bar{R}_c)(x_0^{(i)} - \bar{x}_{0,c}), \qquad G(c) = \frac{\overline{\text{Cov}}}{\text{RMS}(\overline{\text{Cov}})}}. \tag{5}$$

Here $\bar{R}_c$ and $\bar{x}_{0,c}$ are the group means. RMS normalization fixes the guidance magnitude across prompts and reward models, removing differences in reward scale and spread without a guidance-strength coefficient. We apply $G(c)$ to every rollout state s_t of prompt c and freeze the target $v_{\text{tgt}}^{\text{ART}}(s_t) = v_\theta^{(0)}(s_t) + G(c)$, where $v_\theta^{(0)} = v_{\text{old}}$ is the prediction cached during rollout. At $v_\theta = v_\theta^{(0)}$, every state receives an output-space force proportional to $G(c)$, with no NFT consistency term (Equations 6 and 3; Insight 1). Figure 4 summarizes the construction.

4.2 Two-Update Restoration: Drift-Adaptive Stability without EMA Tuning

Instead of tuning an EMA restoration schedule (Section 3.2), we split each rollout batch into two disjoint minibatches and apply one MSE update to each. Let $v_\theta^{(0)}$ and $v_\theta^{(1)}$ denote the velocity fields before and

after the first update. Both updates use the frozen target $v_{\text{tgt}}^{\text{ART}} = v_\theta^{(0)} + G(c)$. For each state s_t, the loss is $\mathcal{L}_\theta(s_t) = w(s_t)\,\text{MSE}(v_\theta(s_t), v_{\text{tgt}}^{\text{ART}}(s_t))$, where $\text{MSE}(a,b) = D^{-1}\|a-b\|_2^2$, D is the number of output coordinates, and w combines timestep weighting and masking. Let $d(s_t) = v_\theta(s_t) - v_\theta^{(0)}(s_t)$ denote the current displacement. The loss and output gradient before each update are:

$$\begin{aligned} \mathcal{L}^{(0)} &= w\,\text{MSE}(v_\theta^{(0)}, v_{\text{tgt}}^{\text{ART}}) = \frac{w}{D}\|G(c)\|_2^2 \\ -\nabla_{v_\theta}\mathcal{L}_\theta\big|_{v_\theta = v_\theta^{(0)}} &= \frac{2w}{D}\,G(c) \end{aligned} \qquad \vdots \qquad \begin{aligned} \mathcal{L}^{(1)} &= w\,\text{MSE}(v_\theta^{(1)}, v_{\text{tgt}}^{\text{ART}}) = \frac{w}{D}\|G(c) - d(s_t)\|_2^2 \\ -\nabla_{v_\theta}\mathcal{L}_\theta\big|_{v_\theta = v_\theta^{(1)}} &= \frac{2w}{D}\big(G(c) - d(s_t)\big). \end{aligned} \tag{6}$$

The second update combines the original guidance $G(c)$ with restoration $-d(s_t)$ toward $v_\theta^{(0)}$. Restoration grows with each state's drift, counteracting large displacements while leaving guidance dominant when drift is small, without additional tuning. In NFT, cross-state cancellation weakens guidance, allowing restoration to dominate and slow training (Section 3.2). DiffusionART instead shares $G(c)$ across each prompt's states, reducing this cancellation and improving stability without sacrificing efficiency (Insight 2; Figure 6).

4.3 Trust-Region Masking for ODE Rollouts

Despite restoration, applying guidance gradient in both updates can produce an excessive policy change within a single rollout batch, potentially destabilizing training. Inspired by PPO clipping [17], we constrain it with a trust-region mask as a stabilizing trick. However, ODE transitions are deterministic, providing no usable finite KL to measure policy change. We address this by following Flow-GRPO's ODE-to-SDE formulation [6] to define reference Gaussian transitions for the current and old policies: $q_\theta(\cdot \mid s_t) = \mathcal{N}(\mu(s_t, v_\theta(s_t)), \sigma_{\text{ref}}^2(t)I)$ and $q_{\text{old}}(\cdot \mid s_t) = \mathcal{N}(\mu(s_t, v_\theta^{(0)}(s_t)), \sigma_{\text{ref}}^2(t)I)$. Based on this, we propose a KL proxy to measure the drift:

$$m(s_t) = \frac{1}{D} D_{\text{KL}}(q_\theta(\cdot \mid s_t)\,\|\,q_{\text{old}}(\cdot \mid s_t)) = \frac{\|\mu(s_t, v_\theta(s_t)) - \mu(s_t, v_\theta^{(0)}(s_t))\|_2^2}{2D\sigma_{\text{ref}}^2(t)}. \tag{7}$$

The proxy uses velocity predictions and rollouts remain ODE trajectories. Here $\mu(s_t, v)$ is the mean of the next latent after one reference SDE step with velocity v, and $\sigma_{\text{ref}}(t)$ is the policy-independent noise scale of that step (formulas in Appendix C.2).

Inspired by Flow-DPPO [18], we mask excessive outward drift while retaining inward corrections. We identify outward updates by a positive inner product between the displacement $d(s_t)$ and the second update's output-space descent direction $G(c) - d(s_t)$:

$$w(s_t) = t\,[1 - \mathbf{1}\{m(s_t) \geq \delta\}\,\mathbf{1}\{\langle d(s_t), G(c) - d(s_t)\rangle > 0\}]\,. \tag{8}$$

The two conditions detect excessive drift and outward updates, respectively. We fix the drift threshold δ to 10^{-5} throughout and use the factor t to emphasize high-noise states.

Algorithm 1 DiffusionART: one training iteration

1: Sample with an ODE solver using $v_\theta^{(0)} = v_\theta$. **Rollout Stage**
   For each prompt c, collect K endpoints with rewards R, states s_t, and predictions $v_\theta^{(0)}(s_t)$.
2: Compute $G(c)$ by Equation 5 for each prompt c.
3: Freeze $v_{\text{tgt}}^{\text{ART}}(s_t) = v_\theta^{(0)}(s_t) + G(c)$ for each state s_t of prompt c.

4: Split the samples into two equal-sized disjoint minibatches B_1, B_2. **Training Stage**
5: For each $B \in \{B_1, B_2\}$, recompute $w(s_t)$ and update θ with one step on
$\mathcal{L}(B;\theta) = \frac{1}{|B|}\sum_{s_t \in B} w(s_t)\,\text{MSE}(v_\theta(s_t), v_{\text{tgt}}^{\text{ART}}(s_t))$.

4.4 Extension to Pairwise Reward Models

Unlike math or code, visual outputs often lack objectively verifiable correctness criteria, making pairwise preferences a natural alternative to scalar scores for reward modeling [19]. **DiffusionART-Pairwise** is a variant of DiffusionART which naturally supports online pairwise preference feedback [19]. For each prompt, we divide K endpoints into $K/2$ disjoint pairs and assign winner/loser labels $(1, 0)$, or $(1/2, 1/2)$ for ties. These labels replace scalar rewards in the otherwise unchanged covariance and training pipeline. Appendix C.3 provides the covariance formulation and pseudocode.

5 Experiments

We study the efficiency, stability, and scalability of DiffusionART, progressing from text-to-image (T2I) generation to larger models and higher-dimensional text-to-video (T2V) and image-to-video (I2V) generation.

5.1 Text-to-Image Generation

Setup. We compare DiffusionART with Flow-GRPO, Flow-GRPO-Fast (CPS) [6,20], DanceGRPO [9], Flow-DPPO [18], and DiffusionNFT [7] with and without EMA, denoted DiffusionNFT (EMA) and DiffusionNFT (on-policy). All methods train LoRA adapters [21] with rank $r = 32$ and scaling factor $\alpha = 64$ at 512×512 resolution, without CFG or reference-policy KL regularization. On SD3.5-M, we use a learning rate of 3×10^{-4} and 10-step rollouts. DiffusionNFT uses the original per-task hyperparameters [7] to maximize its baseline performance.

Single- and multi-reward optimization. We evaluate SD3.5-M [22] on GenEval [23], OCR, and PickScore [19], and further jointly optimize GenEval, PickScore, CLIPScore [24], and HPSv2.1 [25]. Figure 5(a–d) shows that DiffusionART behaves similarly under single- and multi-reward training. Figure 5(d) plots the total reward, with components in Appendix A.4. Without per-task EMA tuning, DiffusionART avoids the severe collapse of DiffusionNFT (on-policy) and achieves training efficiency comparable to DiffusionNFT (EMA). It is also much faster than Flow-GRPO, reaching Flow-GRPO's step-500 reward with $10.0\times$, $5.2\times$, and $9.6\times$ fewer training steps on GenEval, OCR, and PickScore, respectively.

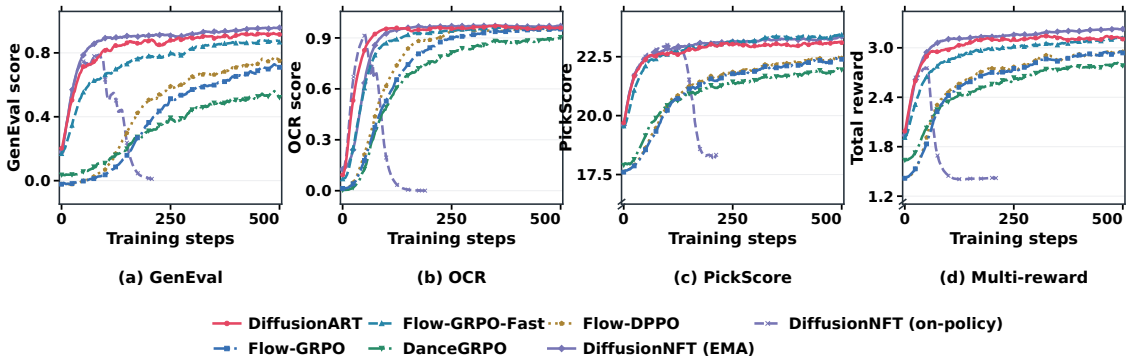

Figure 5: Training rewards: (a–c) single- and (d) multi-reward SD3.5-M.

Scaling to larger models. We extend the same recipe to FLUX.1-dev (12B) [26] on GenEval, using six-step rollouts and its built-in guidance embedding at scale 3.5, without additional CFG. For DiffusionNFT, we retain $\beta = 1$ and rollout-policy EMA decay $\min(0.001n, 0.5)$ from the SD3.5-M GenEval baseline, where n counts successful training updates. As shown in Figure 6(a), DiffusionART learns fastest among all methods, while DiffusionNFT exhibits pronounced training-reward fluctuations because these hyperparameters are not retuned for FLUX. Table 2 in Appendix A.1 reports the GenEval evaluation scores of different methods at step 500.

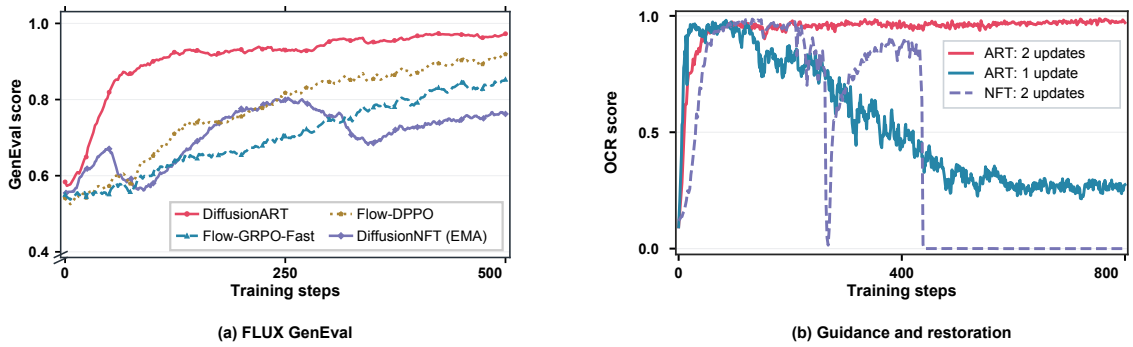

Figure 6: (a) FLUX.1-dev on GenEval. (b) Guidance and restoration ablation on OCR.

Ablation of guidance and restoration. Figure 6(b) compares *ART: 1 update* and *ART: 2 updates*, two DiffusionART variants with the same covariance guidance, against *NFT: 2 updates* with a carefully tuned $\beta = 0.1$. As explained in Appendix B.3, *ART: 1 update* learns quickly but gradually deteriorates. The second update in *ART: 2 updates* introduces restoration and provides long-term stability. In contrast, *NFT: 2 updates* briefly collapses to near zero before recovering. These results highlight the importance of both guidance type and restoration.

5.2 Text-to-Video Generation

Setup. We fine-tune all parameters of the CFG-distilled HunyuanVideo-13B model [27] with a learning rate of 10^{-5} and no KL regularization. Each group-based rollout uses 32 prompts and 8 videos per prompt. We generate 53-frame videos at 480×480 and 8 FPS with 16 sampling steps. VideoReward [28] scores videos at 4 FPS for training and evaluation. All methods use matched TA:VQ:MQ weights of 7:2:1 to improve output quality while preserving prompt following. Figure 7(a–c) compares all three rewards over 400 training steps; Appendix A.5 reports an additional VQ:MQ = 1:1 comparison.

Performance. As shown in Figure 7, DiffusionART surpasses all baselines in both VQ and MQ while keeping TA comparable with the strongest baseline. DiffusionART reaches training VQ of 6 and MQ of 3 after 29 and 46 steps, respectively; Flow-GRPO-Fast, DanceGRPO, and DiffusionNFT (EMA) do not reach these levels within their recorded trajectories. For VQ of 5.5, DiffusionART needs 24 steps and 63.4 GPU hours, versus 273 steps and 1,401.4 GPU hours for DiffusionNFT (EMA): **11.4× fewer training steps and 22.1× fewer GPU hours**. It retains its gains through 400 steps and achieves evaluation VQ and MQ of 8.970 and 4.224, as reported in Table 3. DiffusionART improves evaluation VQ and MQ by 31.4% and 57.4%, respectively, over the strongest baseline.

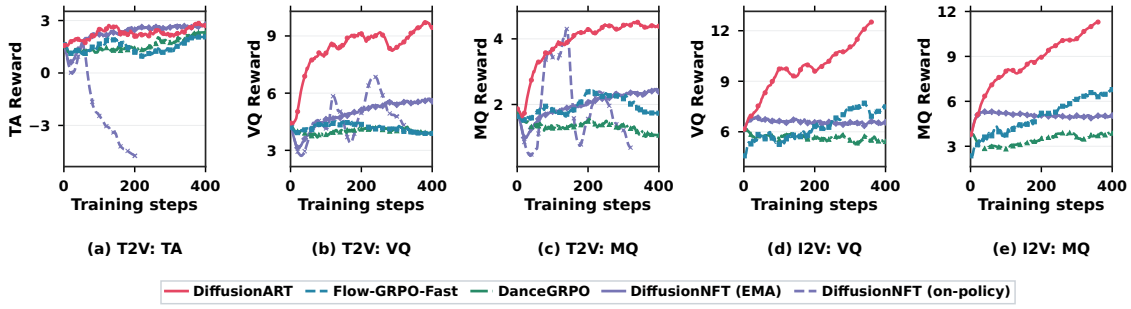

Figure 7: **Efficient reward improvement across video generation.** (a–c) T2V: DiffusionART improves VQ and MQ fastest while maintaining TA comparable to the strongest baseline. (d–e) I2V: DiffusionART rapidly improves both rewards, whereas the baselines improve slowly or plateau.

Ablation of trust-region masking. We remove only the trust-region mask described in Section 4.3, retaining normalized covariance guidance, timestep weighting, and two disjoint MSE updates. With VQ:MQ = 1:1 weights, Figure 8 shows that the no-mask run drops around step 200 before recovering, whereas the masked run improves steadily. Masking helps limit excessive updates, serving as an additional stability safeguard; Appendix A.7 provides further diagnostics of gradients and velocity drift, and Appendix A.6 reports the trust-region ablation on T2I tasks.

Pairwise feedback and ablation of group size. The performance of DiffusionART with pairwise train reward is close to that obtained with scalar feedback, using scalar reward as the ground truth evaluation metric. Meanwhile, DiffusionART is insensitive to the group size K, and a smaller K substantially reduces GPU hours. Table 1 shows that, under the same 400-step budget with pairwise feedback, $K = 2$ reaches a final VQ of 12.235, no lower than 12.092 for $K = 8$. Because $K = 2$ uses only a quarter of the rollout videos and training samples, it consumes only 260.8 GPU hours over the 400 steps, 3.4× fewer than the 883.3 GPU hours of $K = 8$. One pair per prompt thus suffices for stable and compute-efficient optimization in this setting.

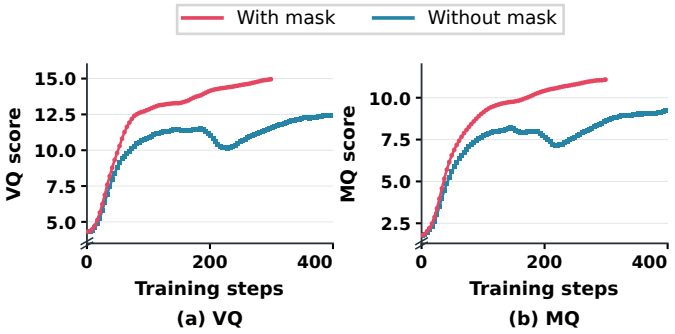

Figure 8: T2V with and without trust-region masking.

Table 1: VQ-only evaluation and GPU hours at step 400.

Method	GPU Hours ↓	VQ ↑
Scalar, $K = 8$	891.1	**13.637**
Pairwise, $K = 2$	**260.8**	12.235
Pairwise, $K = 4$	465.6	10.883
Pairwise, $K = 8$	883.3	12.092

5.3 Image-to-Video Generation

Setup. We fine-tune all parameters of the CFG- and step-distilled Wan2.1-I2V-14B model [29] released by LightX2V [30]. Each rollout uses 32 prompts, FLUX.1-dev reference images, and 8 videos per prompt. We generate 53-frame videos at 400×640 and 8 FPS with four sampling steps. All methods use VQ:MQ $= 1{:}1$ weights, a learning rate of 10^{-5}, and no KL regularization. Reward-model settings follow Section 5.2.

Performance. DiffusionART improves both VQ and MQ efficiently, whereas the baselines improve slowly or plateau (Figure 7(d–e)). It reaches MQ of 6 in 45.8 GPU hours versus 220.3 for Flow-GRPO-Fast, a **4.8× reduction**. DiffusionNFT plateaus early and remains near that plateau through 400 updates, which we attribute to the residual self-consistency force diluting reward-improving guidance after gradient aggregation. Evaluation results are in Table 4.

6 Conclusion

We studied why NFT-style forward-process diffusion post-training can be unstable and found that its objective can introduce an implicit forward–reverse self-consistency force that is not reliably canceled across states, while stronger restoration can improve stability at the cost of slower reward optimization. DiffusionART presents a simple but effective way to systematically cancel the unstable force by aggregating endpoint reward guidance before assigning it to reverse-rollout states, and then using a second update with a trust-region mechanism to provide drift-adaptive restoration for stabilization. Experiments on image and video generation show efficient, stable reward improvement, including with pairwise feedback and small rollout groups. Beyond the algorithm, our work provides insights into forward-process training, offering a clearer understanding of diffusion reinforcement learning and a path toward scalable diffusion post-training for visual generation.

7 Acknowledgement

We thank Kaiwen Zheng and Huayu Chen for the insightful discussion about the intuition behind DiffusionNFT and the valuable suggestion for the experiment design.

References

[1] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[2] Yixian Xu, Kaiyuan Gao, Yuxiang Chen, Yilei Chen, Zecheng Tang, et al. Qwen-Image-2.0-RL technical report. *arXiv preprint arXiv:2606.27608*, 2026.

[3] Yu Gao, Haoyuan Guo, Tuyen Hoang, Weilin Huang, Lu Jiang, et al. Seedance 1.0: Exploring the boundaries of video generation models. *arXiv preprint arXiv:2506.09113*, 2025.

[4] Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. ImageReward: Learning and evaluating human preferences for text-to-image generation. *arXiv preprint arXiv:2304.05977*, 2023.

[5] Bram Wallace, Meihua Dang, Rafael Rafailov, Linqi Zhou, Aaron Lou, Senthil Purushwalkam, Stefano Ermon, Caiming Xiong, Shafiq Joty, and Nikhil Naik. Diffusion model alignment using direct preference optimization. *arXiv preprint arXiv:2311.12908*, 2023.

[6] Jie Liu, Gongye Liu, Jiajun Liang, Yangguang Li, Jiaheng Liu, Xintao Wang, Pengfei Wan, Di Zhang, and Wanli Ouyang. Flow-GRPO: Training flow matching models via online RL. *arXiv preprint arXiv:2505.05470*, 2025.

[7] Kaiwen Zheng, Huayu Chen, Haotian Ye, Haoxiang Wang, Qinsheng Zhang, Kai Jiang, Hang Su, Stefano Ermon, Jun Zhu, and Ming-Yu Liu. DiffusionNFT: Online diffusion reinforcement with forward process. *arXiv preprint arXiv:2509.16117*, 2025.

[8] Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. *arXiv preprint arXiv:2305.13301*, 2023.

[9] Zeyue Xue, Jie Wu, Yu Gao, Fangyuan Kong, Lingting Zhu, et al. DanceGRPO: Unleashing GRPO on visual generation. *arXiv preprint arXiv:2505.07818*, 2025.

[10] Xiaoxuan He, Siming Fu, Yuke Zhao, Wanli Li, Jian Yang, Dacheng Yin, Fengyun Rao, and Bo Zhang. TempFlow-GRPO: When timing matters for GRPO in flow models. *arXiv preprint arXiv:2508.04324*, 2025.

[11] Jing Wang, Jiajun Liang, Jie Liu, Henglin Liu, Gongye Liu, Jun Zheng, Wanyuan Pang, Ao Ma, Zhenyu Xie, Xintao Wang, Meng Wang, Pengfei Wan, and Xiaodan Liang. GRPO-Guard: Mitigating implicit over-optimization in flow matching via regulated clipping. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5988–5998, 2026.

[12] Yixian Xu, Yuanrui Zhang, Shengjie Luo, Liwei Wang, and Di He. Designing reinforcement learning for diffusion models: A unified path-space view. *arXiv preprint arXiv:2608.14430*, 2026.

[13] Yuanzhi Zhu, Xi Wang, Stephane Lathuiliere, and Vicky Kalogeiton. Diffusion reinforcement learning via centered reward distillation. *arXiv preprint arXiv:2603.14128*, 2026.

[14] Shuchen Xue, Chongjian Ge, Shilong Zhang, Yichen Li, and Zhi-Ming Ma. Advantage weighted matching: Aligning rl with pretraining in diffusion models, 2026. URL https://arxiv.org/abs/2509.25050.

[15] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow, 2022. URL https://arxiv.org/abs/2209.03003.

[16] Peter Potaptchik, Cheuk-Kit Lee, and Michael S. Albergo. Tilt matching for scalable sampling and fine-tuning. *arXiv preprint arXiv:2512.21829*, 2025.

[17] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[18] Bowen Ping, Xiangxin Zhou, Penghui Qi, Minnan Luo, Liefeng Bo, and Tianyu Pang. Flow-DPPO: Divergence proximal policy optimization for flow matching models. *arXiv preprint arXiv:2606.11025*, 2026.

[19] Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. *arXiv preprint arXiv:2305.01569*, 2023.

[20] Feng Wang and Zihao Yu. Coefficients-preserving sampling for reinforcement learning with flow matching, 2025. URL https://arxiv.org/abs/2509.05952.

[21] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, et al. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

[22] Stability AI. Stable diffusion 3.5 medium. https://huggingface.co/stabilityai/stable-diffusion-3.5-medium, 2024.

[23] Dhruba Ghosh, Hanna Hajishirzi, and Ludwig Schmidt. GenEval: An object-focused framework for evaluating text-to-image alignment. *arXiv preprint arXiv:2310.11513*, 2023.

[24] Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. CLIPScore: A reference-free evaluation metric for image captioning. *arXiv preprint arXiv:2104.08718*, 2021.

[25] Xiaoshi Wu, Yiming Hao, Keqiang Sun, Yixiong Chen, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score v2: A solid benchmark for evaluating human preferences of text-to-image synthesis. *arXiv preprint arXiv:2306.09341*, 2023.

[26] Black Forest Labs. FLUX.1 [dev]. https://huggingface.co/black-forest-labs/FLUX.1-dev, 2024.

[27] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, et al. HunyuanVideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024.

[28] Jie Liu, Gongye Liu, Jiajun Liang, Ziyang Yuan, Xiaokun Liu, et al. Improving video generation with human feedback. *arXiv preprint arXiv:2501.13918*, 2025.

[29] Wan Team. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[30] LightX2V. Wan2.1-I2V-14B-720P-StepDistill-CfgDistill-Lightx2v. https://huggingface.co/lightx2v/Wan2.1-I2V-14B-720P-StepDistill-CfgDistill-Lightx2v, 2025.

Appendix Contents

A Additional Experimental Results **14**

A.1 Scaling to FLUX.1-dev 14

A.2 T2V Evaluation 14

A.3 I2V Evaluation 14

A.4 Multi-Reward Optimization for T2I 14

A.5 Alternative Reward Weights for T2V 14

A.6 Trust-Region Masking for Image Generation 16

A.7 Trust-Region Masking Diagnostics 16

B Proofs **17**

B.1 Conditional Covariance Identity for Ideal NFT 17

B.2 NFT Force and Restoration Decompositions 17

B.3 Covariance as the First-Order Reward Tilt 19

C Clarification of Methods and Experiments **19**

C.1 Reverse-Process Policy Optimization 19

C.2 Reference Transitions for ODE Trust-Region Masking 19

C.3 Pairwise Feedback: Identity and Implementation 20

D Visualization **21**

A Additional Experimental Results

A.1 Scaling to FLUX.1-dev

Table 2: **FLUX.1-dev GenEval evaluation.**

Method	Eval reward
Flow-GRPO-Fast	0.8374
Flow-DPPO	0.9183
DiffusionNFT (EMA)	0.8490
DiffusionART	**0.9684**

A.2 T2V Evaluation

Table 3 reports the latest evaluation scores under matched TA:VQ:MQ $= 7{:}2{:}1$ training weights. DiffusionART improves evaluation VQ and MQ by 31.4% and 57.4%, respectively, over the strongest baseline.

Table 3: **T2V evaluation.** Matched TA:VQ:MQ $= 7{:}2{:}1$.

Method	VQ ↑	MQ ↑	TA ↑
Pretrained	5.195	1.943	1.118
Flow-GRPO-Fast	3.149	1.874	1.667
DanceGRPO	3.922	1.206	1.685
DiffusionNFT (EMA)	6.827	2.683	2.096
DiffusionNFT (on-policy)	5.543	1.839	-4.480
DiffusionART	**8.970**	**4.224**	**2.111**

A.3 I2V Evaluation

Table 4: **I2V evaluation.** VQ:MQ $= 1{:}1$.

Method	VQ ↑	MQ ↑
Flow-GRPO-Fast	5.771	4.710
DanceGRPO	5.123	3.266
DiffusionNFT (EMA)	5.516	4.161
DiffusionART	**11.196**	**10.476**

A.4 Multi-Reward Optimization for T2I

We jointly optimize GenEval, PickScore, CLIPScore, and HPSv2.1 on a single SD3.5-M policy for 500 training steps. Figure 9 reports each reward separately, comparing how the methods balance compositional accuracy, image–text alignment, and human preference under a shared multi-reward objective.

A.5 Alternative Reward Weights for T2V

Figure 10 compares all methods with VQ:MQ $= 1{:}1$ weights and no TA reward. DiffusionART reaches a training VQ of 6 in 20 steps, compared with 50 for DiffusionNFT (EMA) and 427 for Flow-GRPO-Fast: **2.5× and 21.4× fewer training steps**. It uses 44.2 GPU hours, versus 250.2 and 502.5, giving **5.7× and 11.4× GPU-hour speedups**. DanceGRPO reaches this level after 821 steps and 1805.5 GPU hours. Using the mean raw VQ/MQ reward, $(\mathrm{VQ} + \mathrm{MQ})/2$, DiffusionART reaches the late-stage reward levels of DiffusionNFT (EMA) and Flow-GRPO-Fast with $12.02\times$ and $11.72\times$ fewer GPU hours, respectively (Figure 1). Its mean reward over the final 50 training steps is 13.018, compared with 10.105 and 8.510 for the two baselines, representing improvements of 28.8% and 53.0%.

For MQ of 3, DiffusionART needs 19 steps and 42.9 GPU hours, compared with 33 steps and 165.6 GPU hours for NFT, and 117 steps and 149.4 GPU hours for Fast. This gives **1.7×/6.2× step speedups** and **3.9×/3.5× GPU-hour speedups** over NFT/Fast. Thus, DiffusionART also learns efficiently when the objective focuses only on VQ and MQ.

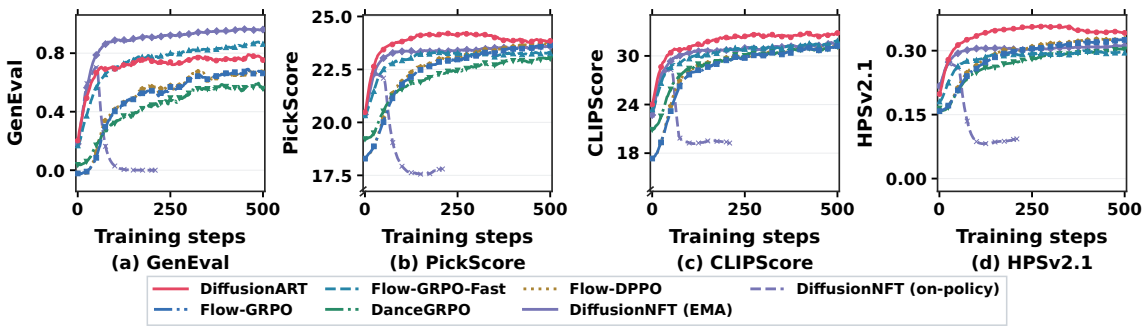

Figure 9: Training reward curves for four jointly optimized objectives on SD3.5-M over 500 steps. PickScore and CLIPScore are shown as unscaled model logits.

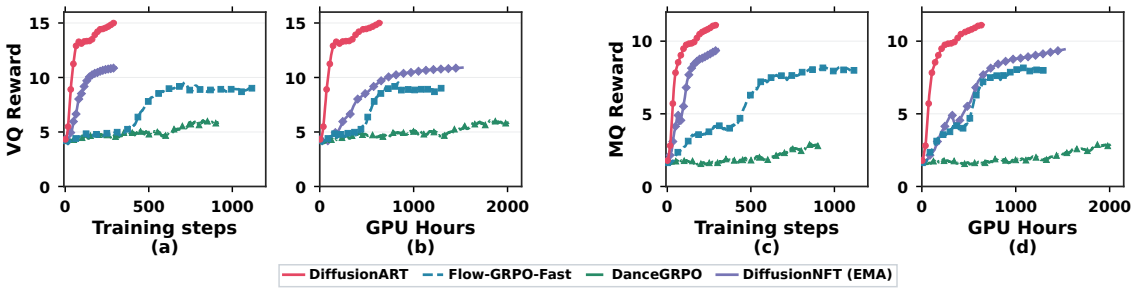

Figure 10: **T2V with matched VQ:MQ** $= 1{:}1$ **weights.** The left pair shows VQ and the right pair MQ, each versus training steps and GPU hours.

Direct optimization with a differentiable reward. We originally followed the undifferentiable reward model setting (i.e. we can only access the scalar reward value, not including the gradient of the reward model). However, methods like ReFL [4] utilize the gradient of a VLM reward model to help training. We also compare DiffusionART with ReFL under matched VQ:MQ $= 1{:}1$ weights (Figure 11). ReFL uses 32 prompts and one video per prompt ($K = 1$), with gradients through the frozen reward model and VAE to the final denoising step; earlier steps are detached. It is trained for 400 updates at a learning rate of 10^{-5}, without KL regularization or EMA.

DiffusionART improves both rewards with fewer optimizer updates, while ReFL uses fewer samples per update ($K = 1$ versus $K = 8$ of DiffusionART) but requires reward-model backpropagation; DiffusionART uses reward values alone.

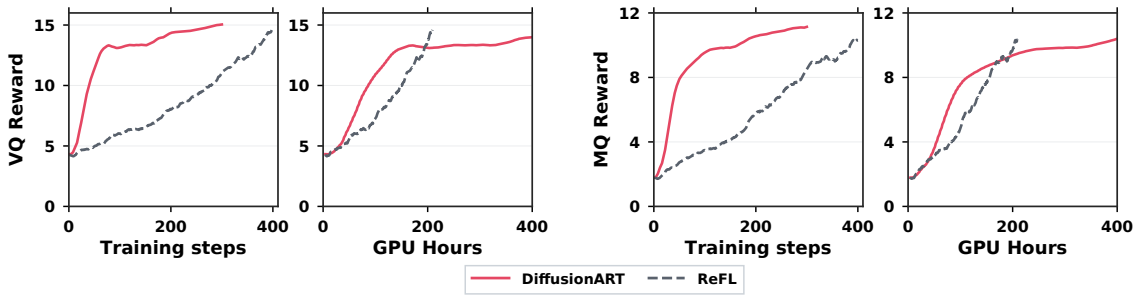

Figure 11: **DiffusionART versus ReFL with matched VQ:MQ** $= 1{:}1$ **weights.** The left pair shows VQ and the right pair MQ, each versus training steps and GPU hours.

A.6 Trust-Region Masking for Image Generation

We remove only the trust-region mask, retaining normalized covariance guidance, timestep weighting, and two disjoint MSE updates. On the three SD3.5-M benchmarks and FLUX.1-dev, training remains effective without masking and can improve slightly faster (Figure 12). On FLUX.1-dev, the no-mask run reaches a training GenEval score of 0.971 by step 180. The benefit of masking is more pronounced for video generation (Figure 8); Appendix A.7 examines its stabilizing effect beyond reward curves.

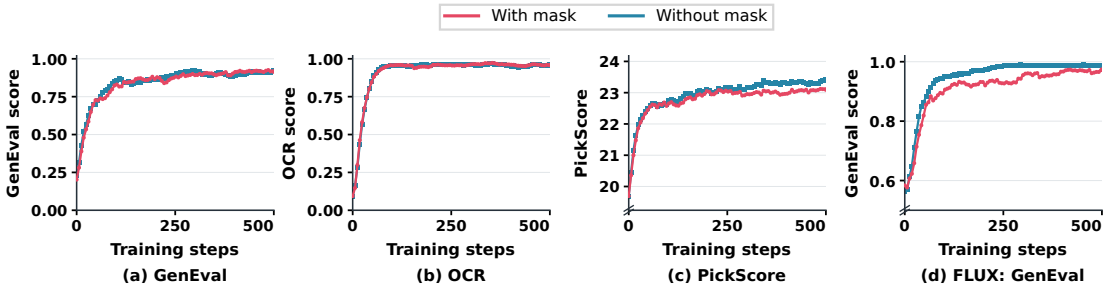

Figure 12: **Image trust-region mask ablation.** (a–c) SD3.5-M; (d) FLUX.1-dev.

A.7 Trust-Region Masking Diagnostics

We further inspect the matched HunyuanVideo comparison with VQ:MQ weights of 1:1. Without masking, training exhibits larger late-stage gradient and velocity-drift spikes (Figure 13), complementing the reward curves in Figure 8(a–b).

Although the mask constrains only the second update of each rollout, its stabilizing effect extends to subsequent first updates. Over the shared first 300 training steps, the maximum first-update gradient norm is 1.96 with masking, compared with 3.33 without it. The velocity drift measured before second updates, which reflects displacement caused by the first update, also shows much larger spikes without masking (Figure 13). With masking, the velocity drift score stays a healthy level. These observations suggest that restricting second updates helps keep later first updates well behaved, rather than merely suppressing gradients at the masked update.

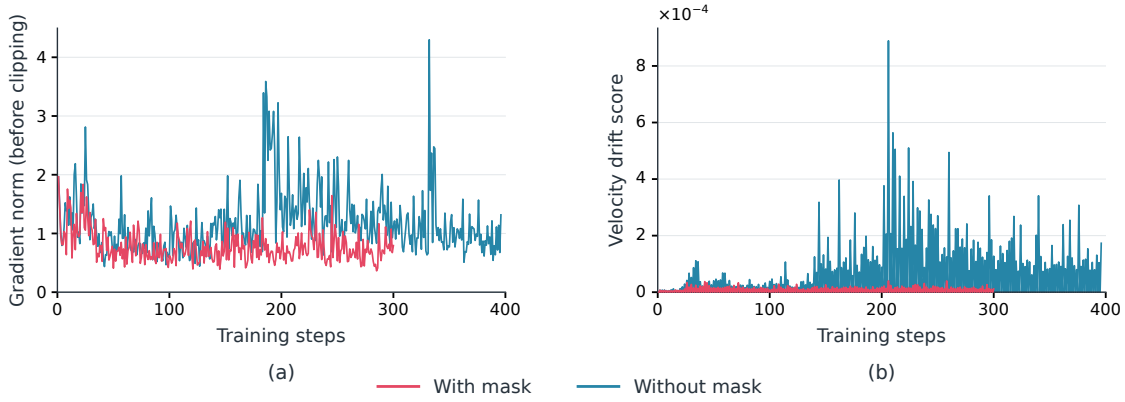

Figure 13: **Trust-region diagnostics on HunyuanVideo-13B.** (a) Parameter gradient norm before clipping. (b) Scheduler-reference-weighted velocity drift energy, averaged over training states before each update.

B Proofs

B.1 Conditional Covariance Identity for Ideal NFT

Lemma 1 (restated). At a fixed supported state s_t, let Q_{old} denote Q_b with $b = \text{old}$, i.e., the joint law obtained by forward noising the rollout endpoints $x_0 \sim p_{\text{old}}(\cdot \mid c)$, and let $q := \mathbb{E}_{Q_{\text{old}}}[r(x_0, c) \mid s_t]$ and $g := \text{Cov}_{Q_{\text{old}}}[r(x_0, c), u_t \mid s_t]$. If the tilted distributions and moments exist and $0 < q < 1$, then

$$q\big(v^{+}_{\text{fwd}} - v^{\text{old}}_{\text{fwd}}\big) = (1-q)\big(v^{\text{old}}_{\text{fwd}} - v^{-}_{\text{fwd}}\big) = g,$$

where all fields are evaluated at s_t. Hence the positive and negative branches point along the same reward-tilt direction g.

Proof. Let $Z_+(c) = \mathbb{E}_{P_{\text{old}}}[r \mid c] > 0$ and $Z_-(c) = \mathbb{E}_{P_{\text{old}}}[1 - r \mid c] > 0$ normalize P_+ and P_-. Since $P_\pm$ reweight only x_0, while ϵ is independent of x_0 and (x_t, u_t) are functions of (x_0, ϵ), the joint laws of $(x_0, \epsilon, x_t, u_t)$ satisfy

$$Q_+(\cdot \mid t, c) = \frac{r(x_0, c)}{Z_+(c)} Q_{\text{old}}(\cdot \mid t, c), \tag{9}$$

$$Q_-(\cdot \mid t, c) = \frac{1 - r(x_0, c)}{Z_-(c)} Q_{\text{old}}(\cdot \mid t, c). \tag{10}$$

Marginalizing onto x_t and using $q = \mathbb{E}_{Q_{\text{old}}}[r(x_0, c) \mid s_t]$ gives

$$Q_+(x_t \mid t, c) = \frac{q}{Z_+(c)} Q_{\text{old}}(x_t \mid t, c), \tag{11}$$

$$Q_-(x_t \mid t, c) = \frac{1 - q}{Z_-(c)} Q_{\text{old}}(x_t \mid t, c). \tag{12}$$

Bayes' rule therefore yields

$$Q_+(\cdot \mid s_t) = \frac{r(x_0, c)}{q} Q_{\text{old}}(\cdot \mid s_t), \tag{13}$$

$$Q_-(\cdot \mid s_t) = \frac{1 - r(x_0, c)}{1 - q} Q_{\text{old}}(\cdot \mid s_t). \tag{14}$$

Taking conditional expectations of u_t gives

$$q v^{+}_{\text{fwd}}(s_t) = \mathbb{E}_{Q_{\text{old}}}[r u_t \mid s_t], \tag{15}$$

$$\big(1 - q\big) v^{-}_{\text{fwd}}(s_t) = \mathbb{E}_{Q_{\text{old}}}[(1 - r) u_t \mid s_t]. \tag{16}$$

Since $v^{\text{old}}_{\text{fwd}}(s_t) = \mathbb{E}_{Q_{\text{old}}}[u_t \mid s_t]$,

$$q\big(v^{+}_{\text{fwd}}(s_t) - v^{\text{old}}_{\text{fwd}}(s_t)\big) = \mathbb{E}_{Q_{\text{old}}}[r u_t \mid s_t] - q\mathbb{E}_{Q_{\text{old}}}[u_t \mid s_t] = g, \tag{17}$$

$$\big(1 - q\big)\big(v^{\text{old}}_{\text{fwd}}(s_t) - v^{-}_{\text{fwd}}(s_t)\big) = \big(1 - q\big)\mathbb{E}_{Q_{\text{old}}}[u_t \mid s_t] - \mathbb{E}_{Q_{\text{old}}}[(1 - r) u_t \mid s_t] = g. \tag{18}$$

This proves both the collinearity and the conditional covariance identity.

B.2 NFT Force and Restoration Decompositions

B.2.1 Equivalent MSE Objective and On-Policy Descent Direction

Preliminary claims (restated). For $\beta > 0$, freeze $\mathcal{D}_{\text{old}}$ and treat r, u_t, and v_{old} as independent of θ. With $v_{\text{tgt}} = v_{\text{old}} + \frac{2r-1}{\beta}(u_t - v_{\text{old}})$, the NFT objective satisfies

$$\mathcal{L}_{\text{NFT}} = \beta^2 \mathbb{E}_{\mathcal{D}_{\text{old}}}\big[\|v_\theta - v_{\text{tgt}}\|_2^2\big] + \text{const},$$

where const is independent of θ. At the on-policy initialization $v_\theta = v_{\text{old}}$, the single-sample output-space descent direction is $-\nabla_{v_\theta} \ell_{\text{NFT}} = 2\beta(2r - 1)(u_t - v_{\text{old}})$.

Proof. Let ℓ_{NFT} denote the weighted positive–negative loss for one sample, so $\mathcal{L}_{\mathrm{NFT}} = \mathbb{E}_{\mathcal{D}_{\mathrm{old}}}[\ell_{\mathrm{NFT}}]$. Write $d = v_\theta - v_{\mathrm{old}}$, $h = u_t - v_{\mathrm{old}}$, and $A = 2r - 1$. Expanding and completing the square gives

$$\begin{aligned}\ell_{\mathrm{NFT}} &= r\|\beta d - h\|_2^2 + (1-r)\|-\beta d - h\|_2^2 \\ &= \beta^2\|d\|_2^2 - 2\beta A\langle d, h\rangle + \|h\|_2^2 \\ &= \beta^2 \left\| d - \frac{A}{\beta} h \right\|_2^2 + (1 - A^2)\|h\|_2^2.\end{aligned}$$

Since $v_{\mathrm{tgt}} = v_{\mathrm{old}} + (A/\beta)h$, the first term is $\beta^2\|v_\theta - v_{\mathrm{tgt}}\|_2^2$. Using $1 - A^2 = 4r(1-r)$ and taking the expectation over the frozen $\mathcal{D}_{\mathrm{old}}$ proves Equation 1, with $\mathrm{const} = \mathbb{E}_{\mathcal{D}_{\mathrm{old}}}[4r(1-r)\|u_t - v_{\mathrm{old}}\|_2^2]$ independent of θ.

Differentiating the same sample loss with respect to v_θ yields

$$-\nabla_{v_\theta}\ell_{\mathrm{NFT}} = 2\beta A h - 2\beta^2 d.$$

At the on-policy initialization $v_\theta = v_{\mathrm{old}}$, we have $d = 0$, so the output-space descent direction is $2\beta(2r - 1)(u_t - v_{\mathrm{old}})$, proving the stated proportionality. □

B.2.2 Reward Guidance and Self-Consistency Force

Proposition 1 (restated). Let $\mathcal{L}_{\mathrm{NFT}}(s_t)$ denote Equation 1 conditioned on s_t, q and g follow Lemma 1, and $e = v^{\mathrm{old}}_{\mathrm{fwd}} - v^{\mathrm{old}}_{\mathrm{rvs}}$. At the pure on-policy update point $v_\theta = v^{\mathrm{old}}_{\mathrm{rvs}}$ and for $\beta > 0$, its normalized output-space force is

$$\mathcal{F}_{\mathrm{NFT}}(s_t) := -\frac{1}{2\beta}\,\nabla_{v_\theta(s_t)}\mathcal{L}_{\mathrm{NFT}}(s_t)\big|_{v_\theta = v^{\mathrm{old}}_{\mathrm{rvs}}} = 2g + (2q - 1)e.$$

Proof. Fix s_t, and write $d = v_\theta(s_t) - v^{\mathrm{old}}_{\mathrm{rvs}}(s_t)$ and $h = u_t - v^{\mathrm{old}}_{\mathrm{rvs}}(s_t)$. The conditional objective is

$$\begin{aligned}\mathcal{L}_{\mathrm{NFT}}(s_t) &= \mathbb{E}_{Q_{\mathrm{old}}}\big[r\|\beta d - h\|_2^2 + (1-r)\|-\beta d - h\|_2^2 \mid s_t\big] &(19)\\ &= \beta^2\|d\|_2^2 - 2\beta\,\langle d, \mathbb{E}_{Q_{\mathrm{old}}}[(2r-1)h \mid s_t]\rangle + \mathbb{E}_{Q_{\mathrm{old}}}[\|h\|_2^2 \mid s_t]. &(20)\end{aligned}$$

At the pure on-policy update point, $d = 0$. Therefore,

$$\begin{aligned}\mathcal{F}_{\mathrm{NFT}}(s_t) &= -\frac{1}{2\beta}\,\nabla_d\mathcal{L}_{\mathrm{NFT}}(s_t)|_{d=0} &(21)\\ &= \mathbb{E}_{Q_{\mathrm{old}}}[(2r-1)h \mid s_t]. &(22)\end{aligned}$$

Because $\mathbb{E}_{Q_{\mathrm{old}}}[h \mid s_t] = v^{\mathrm{old}}_{\mathrm{fwd}} - v^{\mathrm{old}}_{\mathrm{rvs}} = e$ and $\mathrm{Cov}_{Q_{\mathrm{old}}}(r, h \mid s_t) = \mathrm{Cov}_{Q_{\mathrm{old}}}(r, u_t \mid s_t) = g$,

$$\begin{aligned}\mathbb{E}_{Q_{\mathrm{old}}}[(2r-1)h \mid s_t] &= 2\,\mathrm{Cov}_{Q_{\mathrm{old}}}(r, h \mid s_t) + (2\mathbb{E}_{Q_{\mathrm{old}}}[r \mid s_t] - 1)\mathbb{E}_{Q_{\mathrm{old}}}[h \mid s_t] &(23)\\ &= 2g + (2q-1)e, &(24)\end{aligned}$$

which proves Equation 3.

B.2.3 Guidance and Quadratic Restoration

Proposition 2 (restated). Let $d = v_\theta - v_{\mathrm{old}}$ and $h = u_t - v_{\mathrm{old}}$.

$$\mathcal{L}_{\mathrm{NFT}} = \mathbb{E}_{\mathcal{D}_{\mathrm{old}}}\big[-2\beta(2r-1)\langle d, h\rangle + \beta^2\|d\|_2^2\big] + \mathrm{const},$$

where const is independent of the trainable parameters.

Proof. Using the same d and h, the positive and negative residuals are $\beta d - h$ and $-\beta d - h$, respectively. Expanding their weighted squared norms gives

$$r\|\beta d - h\|_2^2 + (1-r)\|-\beta d - h\|_2^2 = \beta^2\|d\|_2^2 - 2\beta(2r-1)\langle d, h\rangle + \|h\|_2^2. \tag{25}$$

Taking $\mathbb{E}_{\mathcal{D}_{\mathrm{old}}}$ and absorbing the final term into the constant proves Equation 4.

B.3 Covariance as the First-Order Reward Tilt

Lemma 2 (restated). If the relevant moments exist and differentiation can be exchanged with expectation near $\tau = 0$, then $\frac{\partial}{\partial \tau}\mathbb{E}_{p_\tau}[x_0 \mid c]\big|_{\tau=0} = \mathrm{Cov}_{p_{\mathrm{old}}}(R, x_0 \mid c)$.

Proof. Fix c and suppress it below. Write $M(\tau) = \mathbb{E}_{p_{\mathrm{old}}}[x_0 e^{\tau R}]$ and $Z(\tau) = \mathbb{E}_{p_{\mathrm{old}}}[e^{\tau R}]$. The normalized tilted density is $p_\tau(x_0) = p_{\mathrm{old}}(x_0)e^{\tau R(x_0)}/Z(\tau)$, so its endpoint mean is $M(\tau)/Z(\tau)$. Assume differentiation under these expectations is valid; a sufficient condition is $\mathbb{E}[(1 + \|x_0\|)(1 + |R|)e^{a|R|}] < \infty$ for some $a > 0$. Since $Z(0) = 1$, the quotient rule gives

$$\frac{\partial}{\partial \tau}\frac{M(\tau)}{Z(\tau)}\bigg|_0 = \mathbb{E}[x_0 R] - \mathbb{E}[x_0]\mathbb{E}[R] = \mathrm{Cov}(R, x_0). \tag{26}$$

From shared guidance to statewise correction. Lemma 2 motivates a shared reward-improving direction computed from the K endpoints already sampled per prompt. This construction avoids state-conditional posterior sampling and excludes the NFT self-consistency term by construction. However, a prompt-level direction need not match the posterior guidance at every state, and different states can respond differently to the same parameter update. Without statewise correction, the resulting deviations can accumulate, providing a explanation for the gradual deterioration with a single update in Figure 6. The second MSE update complements shared guidance with local feedback: its restoring force grows with each state's displacement from the rollout prediction. This controls drift without requiring posterior sampling, combining scalable guidance with statewise stabilization.

C Clarification of Methods and Experiments

C.1 Reverse-Process Policy Optimization

FlowGRPO treats denoising as a finite-horizon policy. For each prompt, it samples K stochastic reverse trajectories, group-normalizes their terminal rewards, and applies clipped PPO to every transition [6]. This directly supervises sampled states but requires tractable transition densities and stored trajectories.

For reverse trajectory $x^{(i)}_{T:0} \sim P^{\mathrm{rev}}_{\mathrm{old}}(\cdot \mid c)$, FlowGRPO assigns the group-normalized terminal advantage A_i to every transition and maximizes

$$\begin{aligned}\mathcal{J}_{\mathrm{FG}} = \mathbb{E}_{i,k}\Big[&\min\big(\rho_{i,k}A_i,\ \mathrm{clip}(\rho_{i,k}, 1-\epsilon_{\mathrm{clip}}, 1+\epsilon_{\mathrm{clip}})A_i\big) \\ &- \lambda_{\mathrm{KL}} D_{\mathrm{KL}}\big(p_\theta(\cdot \mid x^{(i)}_k, c) \,\|\, p_{\mathrm{ref}}(\cdot \mid x^{(i)}_k, c)\big)\Big].\end{aligned} \tag{27}$$

where $\rho_{i,k} = p_\theta(x^{(i)}_{k-1} \mid x^{(i)}_k, c)/p_{\mathrm{old}}(x^{(i)}_{k-1} \mid x^{(i)}_k, c)$ and $\mathbb{E}_{i,k}$ also averages over prompts and old-policy trajectories. Here $\epsilon_{\mathrm{clip}} > 0$ is the PPO clip radius, $\lambda_{\mathrm{KL}} \geq 0$ weights the penalty, and p_{ref} is a fixed reference transition. This requires the stochastic reverse transition used for rollout to expose a tractable density.

C.2 Reference Transitions for ODE Trust-Region Masking

We strictly follow Flow-GRPO's ODE-to-SDE construction [6]; we refer to that work for the derivation. With reverse step size $h = t - t_{\mathrm{next}} > 0$ and state $s_t = (x_t, t, c)$, where x_t is the current latent, the reference transition has mean and variance when $1 > t > 0$:

$$\begin{aligned}\mu(s_t, v) &= x_t + h\left[\left(1 + \frac{\nu^2}{2}\right)v - \frac{\nu^2}{2(1-t)}x_t\right], \\ \sigma^2_{\mathrm{ref}}(t) &= \nu^2\frac{t}{1-t}h.\end{aligned} \tag{28}$$

Substituting $v_\theta(s_t)$ and $v^{(0)}_\theta(s_t)$ gives the current and old reference means, respectively, hence $\mu(s_t, v_\theta(s_t)) - \mu(s_t, v^{(0)}_\theta(s_t)) = h(1 + \nu^2/2)d(s_t)$. At $t = 1$, we use the endpoint convention $\mu(s_t, v) = x_t + hv$ and $\sigma^2_{\mathrm{ref}}(1) = \nu^2$.

We use $\nu = 0.8$ and $\delta = 10^{-5}$ without further tuning. The mask is recomputed before each update and treated as constant during backpropagation.

C.3 Pairwise Feedback: Identity and Implementation

Pairwise covariance identity. For K endpoints forming $M = K/2$ disjoint pairs, with pairwise labels $y_i \in \{0, \frac{1}{2}, 1\}$ replacing R_i in Equation 5, use population reduction $1/K$ globally and $1/2$ within each pair. The global covariance equals the average of the M pair covariances: $\overline{\mathrm{Cov}} = (2K)^{-1} \sum_{\substack{j=1 \\ j \text{ non-tied}}}^{M} (x_{0,j}^{\mathrm{win}} - x_{0,j}^{\mathrm{lose}})$. Equivalently, the coefficient is $1/(4M)$. Tied pairs contribute zero. For $K = 2$, the covariance reduces to one quarter of the winner-minus-loser difference: after normalization, the guidance simply points from the worse sample to the better one. A tie yields zero guidance.

Proof. Every pair has label mean $1/2$, including ties, so the global label mean is also $1/2$. Thus centering endpoints changes no cross term:

$$\overline{\mathrm{Cov}} = \frac{1}{K} \sum_{i=1}^{K} (y_i - \tfrac{1}{2})(x_0^{(i)} - \bar{x}_{0,c}) = \frac{1}{K} \sum_{i=1}^{K} (y_i - \tfrac{1}{2}) x_0^{(i)} \tag{29}$$

$$= \frac{1}{2K} \sum_{\substack{j=1 \\ j \text{ non-tied}}}^{M} \left(x_{0,j}^{\mathrm{win}} - x_{0,j}^{\mathrm{lose}}\right). \tag{30}$$

A non-tied pair's covariance, computed with denominator two, is one quarter of its winner-minus-loser difference. Averaging those pair covariances gives the same expression; tied pairs contribute zero. The pairwise labels enter the same covariance operator as scalar rewards, establishing the stated specialization. The coefficient $\frac{1}{2K}$ doesn't matter since the covariance will be normalized to form the guidance.

Guidance assignment. We compare prompt-global and pair-local guidance on GenEval, using 24 images per prompt to form 12 disjoint pairs. Prompt-global guidance aggregates pair covariances before normalization and applies the shared direction to all trajectories. Pair-local guidance normalizes each pair's covariance separately and applies it only to that pair. At step 400, prompt-global guidance reaches a held-out GenEval score of **0.91**, versus **0.78** for pair-local guidance. It also achieves higher training rewards (Figure 14), showing that the better practice is pooling the pairs of the same prompt together before computing the covariance to reduce the variance.

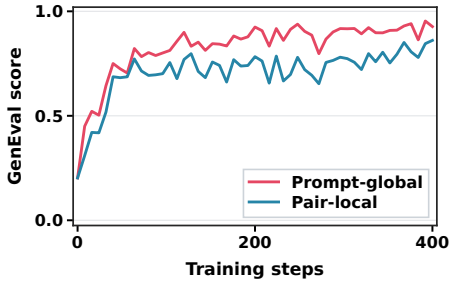

Figure 14: Guidance assignment on GenEval.

Algorithm 2 DiffusionART-Pairwise: one training iteration

1: Sample with an ODE solver using $v_\theta^{(0)} = v_\theta$. **Rollout Stage**
For each prompt c, collect an even number K of endpoints $x_0^{(i)}$, states s_t, and predictions $v_\theta^{(0)}(s_t)$.
2: Form $M = K/2$ disjoint pairs per prompt; query the pairwise reward model.
3: Assign pairwise rewards $y_i = 1$ to winners, $y_i = 0$ to losers, and $y_i = \frac{1}{2}$ for ties.
4: Compute covariance guidance by Equation 5, replacing scalar rewards R_i with pairwise rewards y_i.
5: Freeze $v_{\mathrm{tgt}}(s_t) = v_\theta^{(0)}(s_t) + G(c)$ for each state.

6: Split the samples into two equal-sized disjoint minibatches B_1, B_2. **Training Stage**
7: For each $B \in \{B_1, B_2\}$, recompute $w(s_t)$ and update θ with one step on
$\mathcal{L}(B;\theta) = \frac{1}{|B|} \sum_{s_t \in B} w(s_t)\, \mathrm{MSE}(v_\theta(s_t), v_{\mathrm{tgt}}(s_t))$.

D Visualization

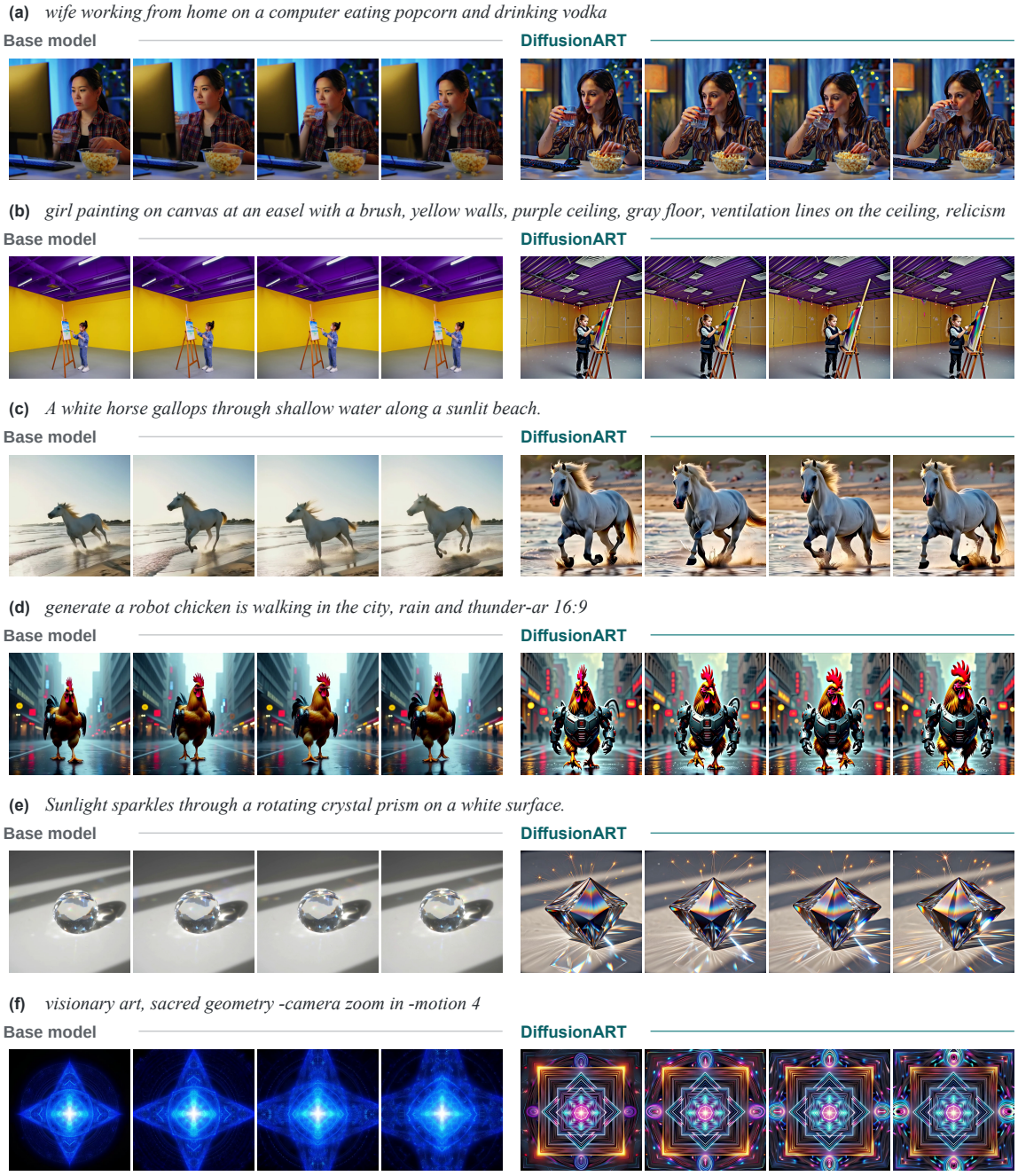

Figure 15: **Text-to-video generation before and after DiffusionART.** Each pair uses the same prompt, seed, and sampling settings. Four chronologically ordered frames are shown at identical frame positions within each pair: pretrained outputs on the left and DiffusionART outputs on the right.

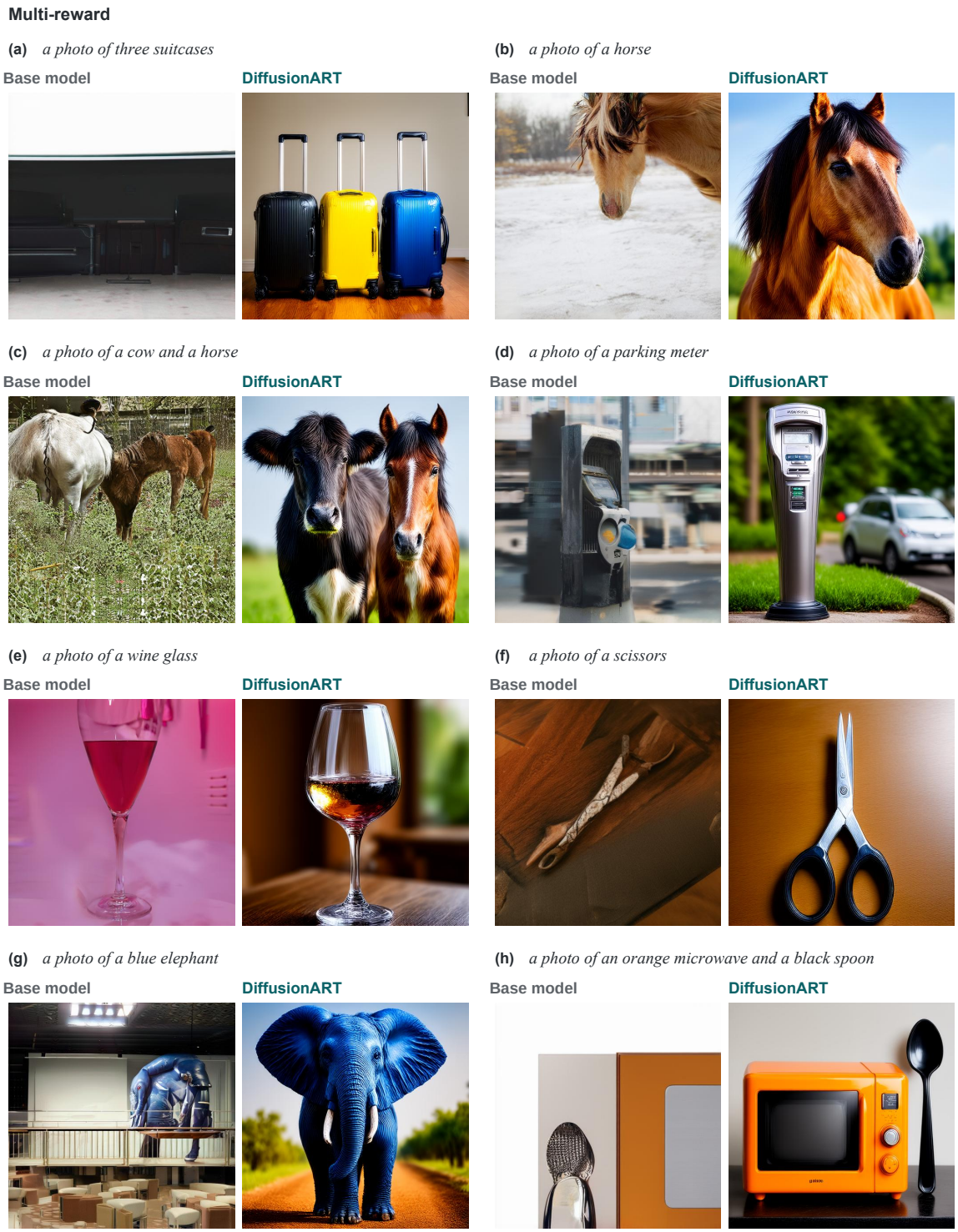

Figure 16: **Text-to-image generation with multi-reward optimization.** Selected SD3.5-M outputs before and after DiffusionART, jointly optimized with GenEval, CLIPScore, PickScore, and HPSv2. Each pair shows the same prompt, with pretrained outputs on the left and DiffusionART outputs on the right.

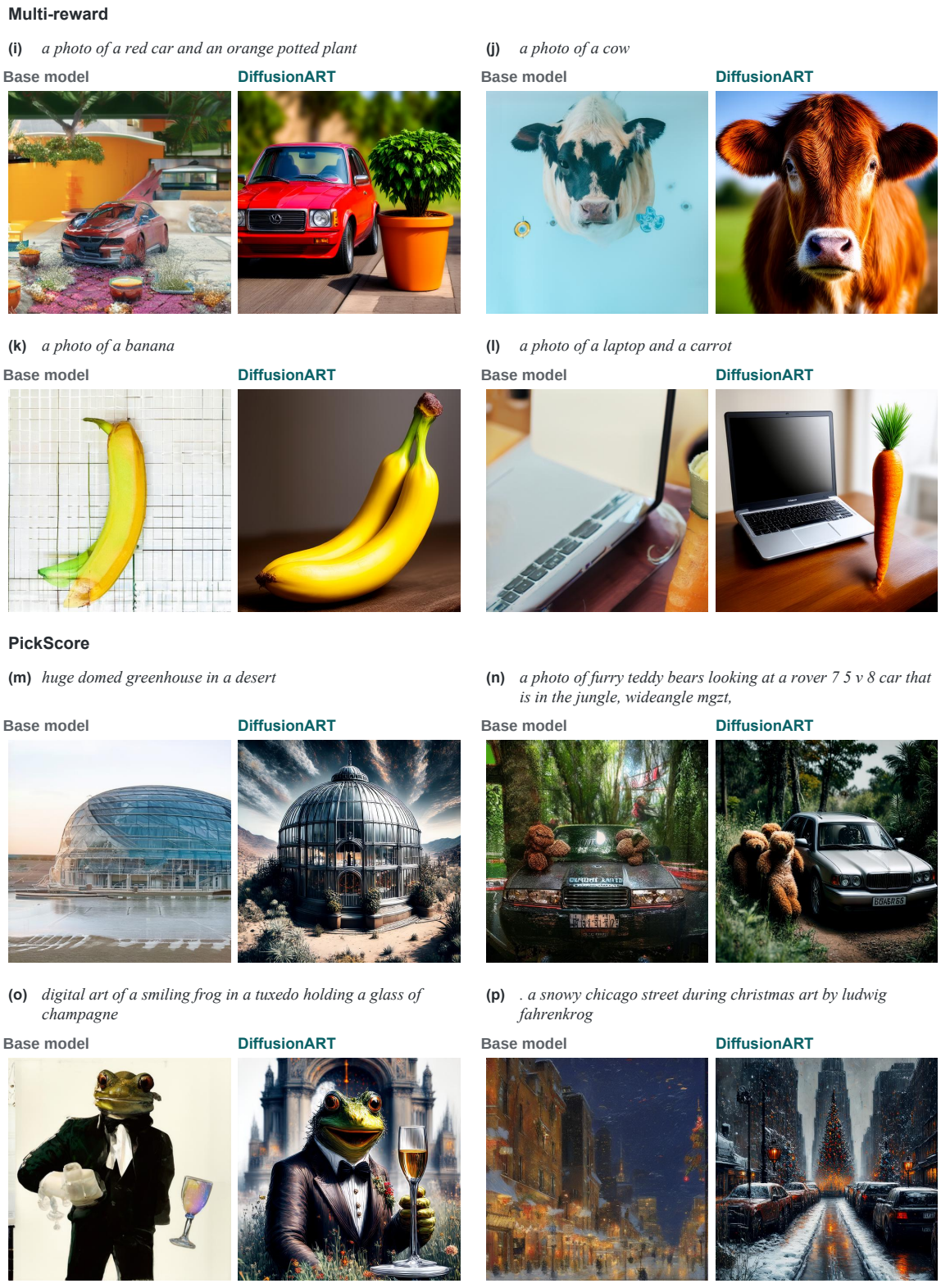

Figure 17: **Additional text-to-image comparisons.** The top two rows use multi-reward optimization; the bottom two use PickScore. Each pair shows pretrained and DiffusionART outputs under the same prompt.